GreenSettle

Carbon credits for cloud jobs, settled **after** the run from **measured** energy, with honest error bars

TODO: name · course · university

Problem

Carbon-aware scheduling moves jobs to greener hours, but credits are usually granted from forecasts and estimates. Nobody checks afterwards whether a job really saved anything, so noise and forecast errors can earn rewards.

Idea in three lines

1. Defer a job to the greenest slot in its wait window, only if the forecast gain beats its noise.
2. Measure the job's CPU energy, with uncertainty, while it runs.
3. Settle after the run against the same job started at submission (actual grid data); pay only for savings beyond the noise.

Contribution

What may be new here: ex-post counterfactual settlement with propagated measurement uncertainty and a dead zone.

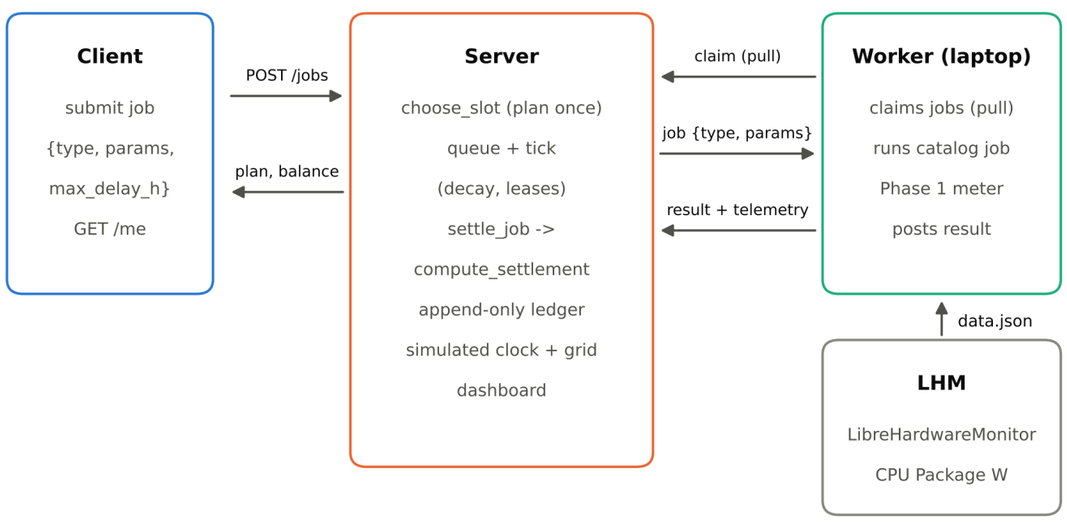
Only a limited literature search was done; related work is incomplete.

What is simulated

- the grid carbon-intensity curve (SimulatedCurve, seed 42)
- demo overrides (green/dirty) on that curve
- the forecast and its error model
- the measurement-noise model in the gaming table and E4
- job energies in E3, E4 and the gaming table

Real: CPU package power (LibreHardwareMonitor) and job CPU time (psutil), one laptop.

Architecture

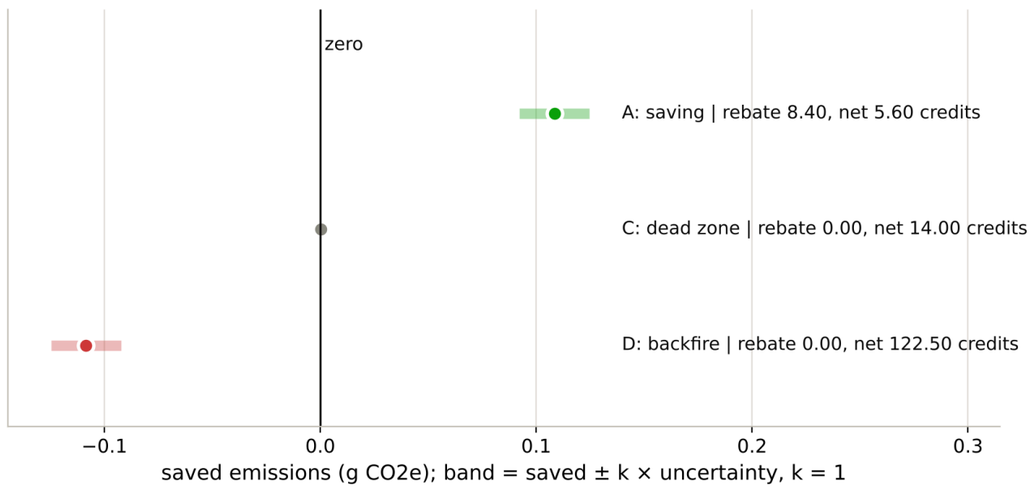


Architecture: the worker on the laptop pulls jobs; the server plans with choose_slot and settles measured energy into an append-only ledger.

The settlement rule in four lines

```
fee   = p·E·i_run
saved = E·(i_submit − i_run) ± σ
if saved − kσ > 0: rebate = min(ρ·p·(saved − kσ), γ·fee)
net   = fee + surcharge − rebate ≥ (1−γ)·fee
```

σ combines the energy and both intensity errors in quadrature. Inside the dead zone (|saved| ≤ kσ) nothing is paid back or added. i_submit is the counterfactual from actual data; a surcharge applies only to a proven backfire and is off by default.



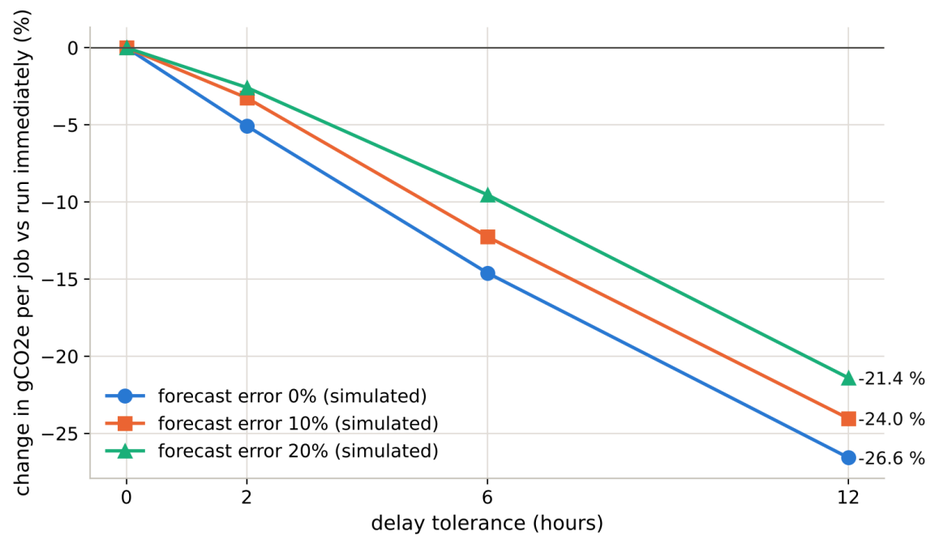
Dead zone: a rebate is paid only if the whole band lies above zero, and a backfire is declared only if it lies below zero (worked examples A, C, D, computed by compute_settlement).

Limits

- CPU package only: RAM, screen, disk and network are not measured.
- CPU-time attribution ignores that P-core and E-core seconds cost different energy.
- The server trusts the worker's measurements (no attestation).
- Jobs are planned once; failed jobs are not settled (their energy is unbilled).
- One mean intensity per job; one laptop; all parameters are placeholders.

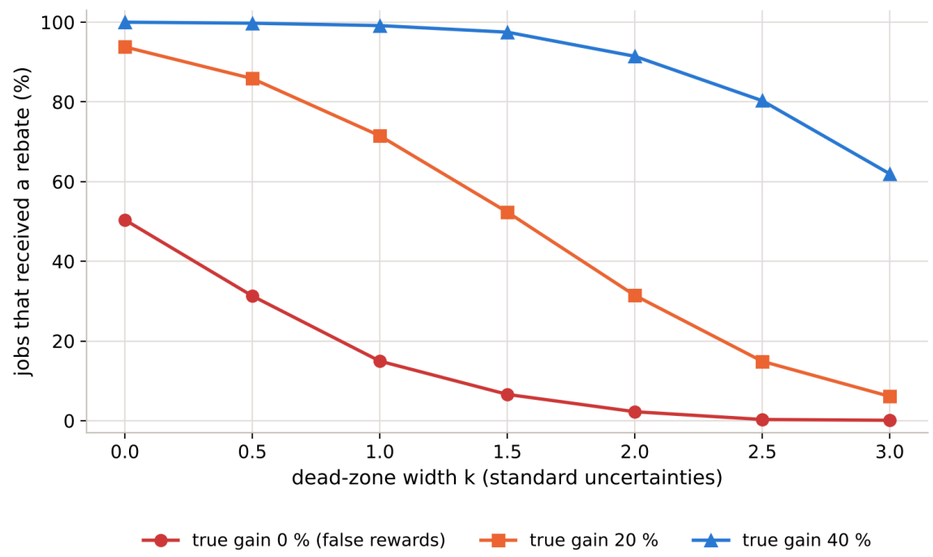
Results

simulated unless stated



E3 (simulated grid and forecast): emissions per job relative to running immediately, choose_slot with defer_z = 1, 504 jobs over 7 days.

> E3 (simulated grid and forecast, 10 % forecast error, z = 1): -12.3 % gCO2e per job with 6 h tolerance (mean wait 2.1 h), -24.0 % with 12 h; 15% and 6% of deferrals backfired.



E4 (simulated 10 % measurement noise, 2000 jobs per point): a wider dead zone removes false rewards faster than deserved ones.

> E4 (simulated 10 % noise): at k = 1 a rebate went to 14.9% of jobs with no real saving (k = 0: 50.4%), and to 71.5% / 99.2% of jobs with a true 20 % / 40 % saving.

> Gaming table (simulated): no ex-post job earned credits (lowest net 0.20 credits); the simplified ex-ante strawman paid 2 job(s) to run.

> **E1 repeatability on the real laptop: PENDING (run on the laptop).**

Placeholders

Sensor tolerance	5% of package energy
Attribution coefficients	base 0.05, k 0.2
Intensity uncertainty	10% relative (simulated curve)
Forecast error model	5% + 2% per hour of lead, max 40%, bias 0
Tier thresholds	red < 20, standard 20-100, green >= 100 credits; priority 0/1/2; max running 1/1/2
p (credits per g CO2e)	1000
rho (rebate share)	0.5
gamma (rebate cap / fee)	0.6
k (dead-zone width)	1
sigma (backfire surcharge)	0
Decay half-life	30 days
Monthly allowance	100 credits